CODEASTRA 2.0 · CLOUD COMPUTING & DISTRIBUTED SYSTEMS

# Geo-Distributed Disaster Data Replication with Graceful Degradation

A multi-region report service that keeps accepting data when the network breaks, merges safely when it returns, and tells operators exactly what is at risk.

| 3 | 58 | 0 | <1 s |
|---|---|---|---|
| replicas live on AWS (Mumbai, Hyderabad, Singapore) | automated tests passing | acknowledged writes lost in fault tests | to converge after healing, across 3 AWS regions |

---

Project report for judges and mentors · 12 October 2026
Status: engine complete and verified locally and on a three-region AWS deployment; baseline benchmark and dashboard in progress

# 1. Executive summary

During a disaster, the region that most needs to record reports is often the one that has lost its connection to everyone else. Most cloud databases force a bad choice: **refuse writes** until the network returns, or **accept writes and silently overwrite** whichever edit has the older timestamp.

We are building a replicated report service that takes a third path:

- **Always accept** a report locally, instantly, even when cut off.
- **Never silently lose** a write. Conflicts are kept in a visible ledger and resolved by clear rules or a human operator.
- **Tell the truth** about safety: every report is marked *local*, *replicated* or *confirmed*.
- **Prove it** with an automated fault-injection tester and a side-by-side comparison against a last-writer-wins baseline.

> **One-sentence pitch.** Cloud providers move data between regions; we make that replication trustworthy and understandable during a disaster, by keeping every write, exposing every conflict, and showing what is still at risk.

### Where we are today

The replication engine is built and verified: three nodes replicate, survive partitions, crashes and wrong clocks, and converge to identical state. It runs in Docker and is deployed as three servers on AWS (Mumbai, Hyderabad, Singapore), where the full partition, heal and crash-recovery scenario passes 12 of 12 checks. The comparison baseline with its benchmark numbers and the operator dashboard are the remaining work.

## 2. The problem we were given

**Problem statement (Cloud Computing & Distributed Systems, #2):** critical systems cannot assume a single region or network path is always available. During a disaster the system may need to keep accepting reports while disconnected from its primary service and later reconcile conflicting updates.

| What the brief asks for | Our response |
|---|---|
| Two or more replicas, local writes, persistence | Three replicas, each its own process; every node writes to its own SQLite event log first |
| Replication protocol and recovery | Pull-based version-vector sync every 0.5 s, relaying through third nodes, catch-up after a heal or crash |
| Partition simulation | Per-link block, node pause (crash), clock skew and reset through an admin API |
| Defined conflict-resolution policy | Per-field merge rules plus a conflict ledger and operator decisions |
| Consistency-status API | `/status`, per-report durability, `/state-hash` |
| Operational dashboard | Web dashboard (in progress, built by a teammate) |
| "Explicitly choose and defend your consistency model" | Section 4 |

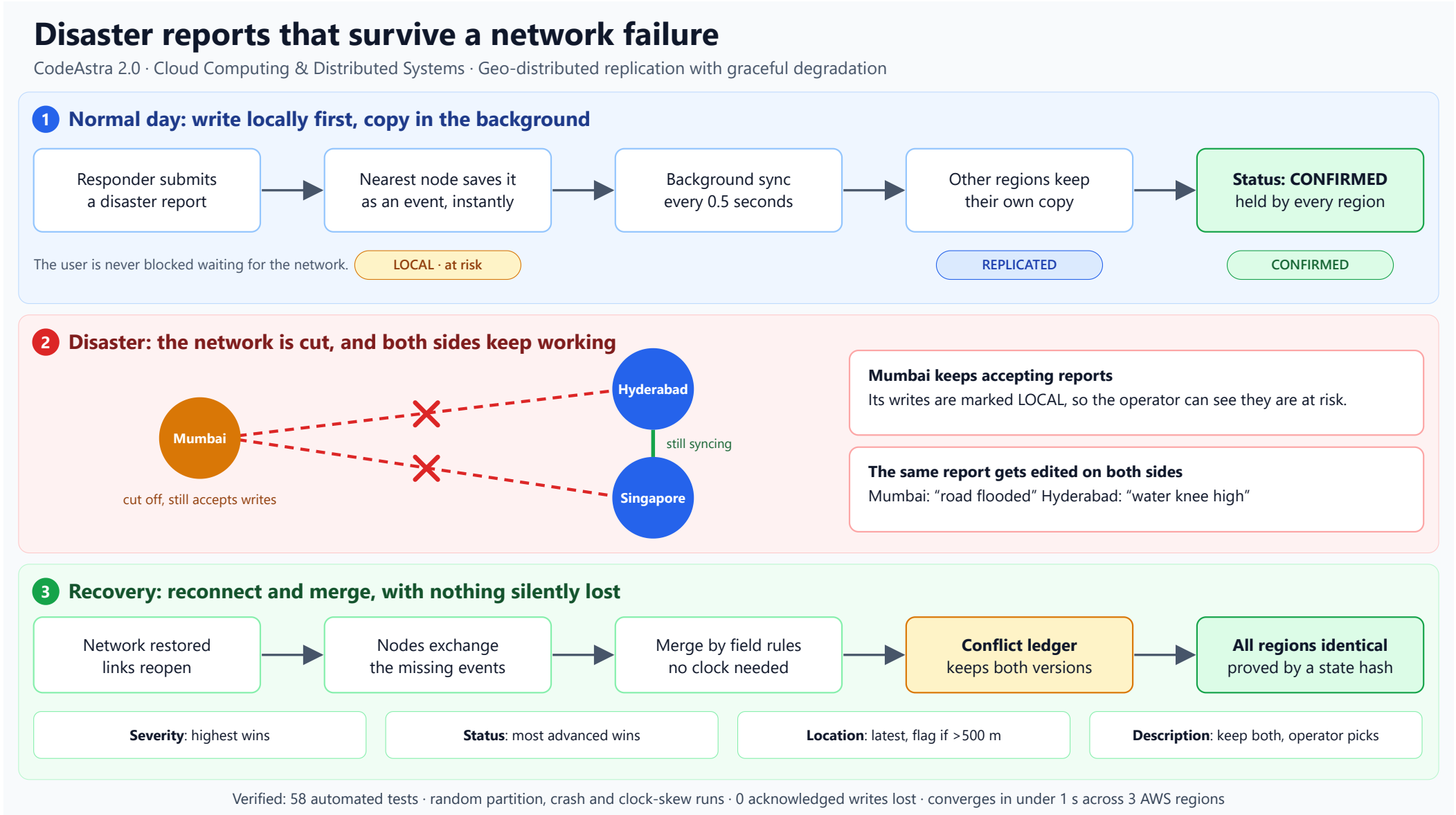



Figure 1. End-to-end workflow: normal operation, disaster partition, recovery and merge.

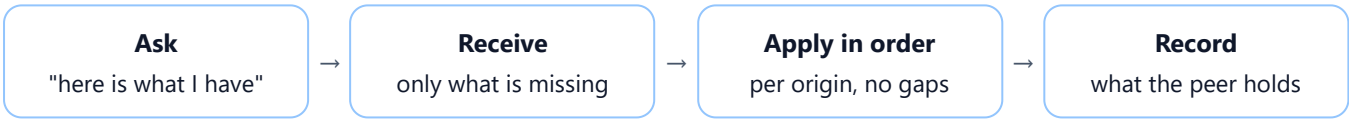
## 3. How it works

### 3.1 Inside one node




- **Events, not rows.** Every create or edit is stored as an immutable event. A report's current state is *calculated* from its events. Nothing is edited or deleted, so nothing can be silently overwritten.
- **Hybrid logical clocks.** Each event carries a clock value that never goes backwards and is advanced by events received from other nodes, so a node with a wrong wall clock cannot corrupt ordering.
- **Version vectors.** Each event records what its author had already seen. This tells "one edit after another" apart from "two edits at the same time", which timestamps alone cannot do.
- **Pure merge function.** The merged result depends only on the *set* of events, not the order they arrived in, so any two nodes holding the same events compute the same state.

### 3.2 Replication




- Each node pulls from every peer every 0.5 s. A dead peer never delays the others (one thread per peer).
- Applying an event twice does nothing; events arriving out of order or with gaps are skipped and refetched.
- Nodes **relay** for each other: if Mumbai cannot reach Hyderabad but both reach Singapore, they still converge.
- Long outages catch up in batches of 500 events, always a gap-free prefix.
- **Reset epochs** stop a freshly wiped node from receiving old data back from a peer not yet wiped.

### 3.3 Merge rules (the consistency policy)

| Field | Two edits at the same time | Why this rule |
|---|---|---|
| Severity (1-5) | Highest value wins; lowering needs an operator | In a disaster, under-reporting is more dangerous than over-reporting |
| Status | Most advanced wins (open < acknowledged < in progress < resolved); reopening needs an operator | Work already done should not be silently undone by a stale update |
| Location | Latest wins; if the two are more than 500 m apart, flag for an operator | Small corrections are safe; large jumps need a human |
| Description | Keep both versions and ask an operator | Free text cannot be merged safely by a rule |
| Type, reporter | Fixed at creation | Identity of a report must not drift |

If a later edit was written by someone who had already seen both versions, it simply supersedes them. Every concurrent disagreement stays in the ledger as history, whether a rule or a person resolved it.

### 3.4 Durability: telling the truth about risk

| Status | Meaning |
|---|---|
| **local** | Only this node holds the report. If this node is lost, the report is lost. |
| **replicated** | At least one other node is known to hold every event of the report. |
| **confirmed** | Every other node is known to hold it. |

### 3.5 Verification built into the system

Each node exposes a hash of its merged state and the list of event ids it holds. Equal hashes mean two nodes hold the same events and merged them identically. An automated tester uses these to check, after every fault run, that nothing was lost and everything converged.

## 4. Consistency model and why we chose it

### Our choice: available, eventually consistent

- Every node always accepts writes (availability under partition).
- All nodes converge to the same state once they exchange events (**strong eventual consistency**).
- No acknowledged write is ever discarded; disagreements are surfaced, not hidden.

### Why not strong consistency?

- A strongly consistent system must refuse writes on the side without a majority.
- In a disaster that is exactly the side that needs to keep recording reports.
- Reports are mostly new facts that accumulate, so merging is practical and safe.

> **The honest limit.** A report saved in a region that is destroyed before it syncs is lost. No design removes this. We shrink the window to about half a second and show operators which reports are still *local*, so the risk is visible instead of hidden.

## 5. Existing approaches and where they fall short

Evidence level: **verified** means confirmed in documentation or papers during our research; **general knowledge** means widely known but not re-verified by us.

| Approach | What it does | Where it falls short for this problem | Evidence |
|---|---|---|---|
| **DynamoDB Global Tables, eventual mode** | An isolated region keeps accepting writes and catches up when healthy | Conflicts resolved by **last writer wins**, so one edit is silently discarded; depends on clocks | **verified** |
| **DynamoDB Global Tables, strong mode** | A write is confirmed only after another region has it | Exactly 3 regions (or 2 plus a witness), no transactions, higher write latency; an isolated region cannot get confirmation (our inference) | **verified (partition behaviour inferred)** |
| **AWS Amplify DataStore / AppSync** | Offline-first client sync with Auto Merge, optimistic concurrency or a custom Lambda | Auto Merge prefers the server value for single fields; custom logic is left to the developer; syncs devices to one cloud endpoint; Gen 1 reaches end of life on 1 May 2027 | **verified** |
| **Consensus databases (Spanner, CockroachDB, etcd)** | Strong consistency through majority agreement | The minority side of a partition cannot commit writes | **general knowledge** |
| **Cassandra, Riak, CouchDB** | Always-writable replicas | Typically timestamp-based or revision-based winners that can hide the losing version | **general knowledge** |
| **CRDT libraries (e.g. Automerge)** | Replicas converge automatically, peer to peer, with no central server | Built for collaborative documents; no domain rules (such as severity can only escalate), no operator workflow, no durability status | **verified (domain gap is our assessment)** |
| **Real-world track record (Jepsen)** | Independent fault-injection testing of databases | Found split-brain, lost acknowledged writes and lost updates in production systems, so "it replicates" is not proof | **verified** |

> **The gap we target:** AWS and others move the data. They do not decide what is true when two regions disagree, do not tell the operator which reports are at risk, and do not ship their own proof that nothing was lost.

## 6. What is different and innovative

We want to be clear about what is new. Event logs, hybrid clocks and version vectors are established techniques. Our contribution is combining them into a disaster-operations service with behaviour and verification that general-purpose tools do not provide.

| Idea | What it gives the user |
|---|---|
| **Domain-aware merge rules** | Safe defaults for disaster reports: severity escalates, status advances, location jumps are flagged. Not a blanket "latest wins". |
| **Visible conflict ledger** | Every concurrent disagreement is stored with both versions and who decided. Nothing disappears silently. |
| **Per-report durability** | Operators see which individual reports are still at risk, not just a table-level latency number. |
| **Causality, not clocks** | Correct behaviour even when a node's clock is an hour wrong. We test this explicitly. |
| **Built-in proof** | A chaos tester runs random partitions, crashes and clock skew, then asserts convergence and zero lost writes, and compares against a last-writer-wins baseline. |
| **Safe reset across nodes** | Epoch numbers prevent old data reappearing after a cluster reset, a real operational trap. |

## 7. Technology stack

| Layer | Choice | Reason |
|---|---|---|
| Language | Python 3.13 | Fast to build and test; strong ecosystem for fault-injection scripts |
| API server | FastAPI + Uvicorn | Typed requests, automatic interactive docs |
| Storage | SQLite (one file per node) | Durable across crashes, zero setup; fits the simulated-region scope |
| Node-to-node calls | httpx | Simple HTTP, easy to block and inspect in tests |
| Tests | pytest | Unit, multi-node and random-fault tests |
| Packaging | Docker and Compose | One command for teammates; same image locally and in the cloud |
| Cloud | AWS EC2 in three regions (deployed) | Real geographic separation. One small instance per region, created and removed by one script, all resources tagged for clean teardown |
| Access control | Shared-secret header on admin, sync and operator endpoints | Lets the nodes be reachable from any network while fault injection and operator decisions stay protected; reporting and reading stay open |
| Dashboard | Web single-page app | Shows regions, reports, conflicts and fault controls |

We deliberately did not implement Raft or Paxos: this problem calls for an available design, and consensus would reintroduce the exact failure we are avoiding.

## 8. Progress so far

| Component | Status | Evidence |
|---|---|---|
| Event log, logical clock, create and edit API | **Done** | Unit and API tests |
| Merge engine and per-field rules | **Done** | One test per rule, plus 40 random histories |
| Conflict ledger and operator resolve | **Done** | API and replication tests |
| Sync between nodes, relaying, batching | **Done** | Multi-node tests |
| Durability states, peer status, state hash | **Done** | Multi-node tests |
| Fault injection (block, pause, clock, reset) | **Done** | Admin API and tests |
| Local three-node cluster and smoke test | **Done** | 12 of 12 checks over real HTTP |
| Docker packaging | **Done** | Image and compose file; data survives a hard kill and a full restart |
| Admin token on sensitive endpoints | **Done** | Tests for open, protected and wrong-token cases; verified live on AWS |
| AWS three-region deployment | **Done** | Mumbai, Hyderabad and Singapore; 12 of 12 checks across the real regions |
| Last-writer-wins baseline and chaos benchmark | **In progress** | Assigned to a teammate; results file format agreed |
| Operator dashboard | **In progress** | Assigned to a teammate; built against the API contract |

### Measured results

| Check | Local cluster | Three AWS regions |
|---|---|---|
| Automated tests | 58 passed | |
| Random fault runs (writes, partitions, crashes, clock skew) | 25 seeds: always converged, no acknowledged write lost, identical ledgers | |
| End-to-end smoke test (replicate, partition, heal, crash) | 12 of 12 passed | 12 of 12 passed |
| Report appears in another region | 0.3 to 0.5 s | 0.17 s |
| Report reaches "confirmed" everywhere | about 0.5 s | 0.29 s |
| Convergence after a partition heals | 0.2 to 0.35 s | 0.56 s |
| Crashed node catches up after resuming | 0.1 to 0.3 s | 0.40 s |

Timings are measured from a laptop that polls the nodes over HTTP, so they include the laptop's own network round trip and a 0.1 s polling step. Read them as upper bounds on the replication delay, from a single run each, not as a statistical benchmark.

## 9. How we will prove it to the judges

| Method | What it shows |
|---|---|
| **1. Live partition demo** | Cut a region, edit the same report on both sides, restore, show the merge and ledger. |
| **2. Side-by-side baseline** | The same scenario on a last-writer-wins cluster: an edit disappears there, but not in ours. |
| **3. Automated chaos benchmark** | Hundreds of seeded random runs with a pass/fail table and charts. |
| **4. Real multi-region deployment** | Three AWS regions, with the partition triggered live. |
| **5. Written model defense** | This report, section 4: why availability, what is lost, and what is not. |

### Invariants checked after every fault run

| ID | Invariant | How it is checked |
|---|---|---|
| I1 | All nodes converge to the same state | Equal state hashes |
| I2 | No acknowledged write is lost | Every acknowledged event id exists on every node |
| I3 | No silent overwrite | Each concurrent disagreement is covered by a rule or a ledger entry |
| I4 | Status never moves backwards without an operator | Compare final status with all writes |
| I5 | Severity never decreases without an operator | Compare final severity with all writes |
| I6 | Convergence time | Measured from heal to equal hashes (p50 and p95) |

The headline figure we expect to present is the **silent-overwrite rate**: the percentage of concurrent edits that vanish in the baseline versus in our system. This number has not been measured yet, so it is not claimed in this report.

### Demo script (3 minutes)

1. Two regions show the same board. Create a report in Mumbai; it appears in Hyderabad and turns green.
2. Cut the link. The regions panel shows the break.
3. Edit the same report on both sides. Badges show "saved locally, at risk".
4. Restore the link. Banner changes from "out of sync" to "in sync".
5. Open the conflict ledger and show both versions and the policy.
6. Switch to side-by-side and show the baseline losing an edit.
7. Run the automated test and show it pass.

## 10. Plan from here

| # | Task | Owner | Done when |
|---|---|---|---|
| 1 | Docker image and Compose for the three nodes | Engine / infra | **Done: smoke test passes against containers** |
| 2 | Connect baseline and dashboard to the engine | All | Dashboard drives the real cluster |
| 3 | Run the chaos benchmark on both clusters | Testing teammate | Results file and two charts produced |
| 4 | Deploy three nodes to AWS regions | Engine / infra | **Done: partition and heal verified across regions** |
| 5 | Rehearse the demo twice, with a laptop-only fallback | All | Runs without venue Wi-Fi |
| 6 | Feature freeze, final report and slides | All | Submission ready |

### Stretch goals if time allows

- **Duplicate-report detection** (same place, time and type), filling the `possible_duplicates` field that is currently empty.
- **Signed node-to-node messages**, to show how the cybersecurity convergence from the brief would plug in.
- **Live updates** to the dashboard through server-sent events instead of polling.

## 11. Risks and limitations

| Item | Mitigation or honest position |
|---|---|
| Data lost if a region is destroyed before syncing | Shown as *local* to operators; sync window about 0.5 s. Cannot be eliminated. |
| Integration of work from several people | Shared API contract, mock-first development, fixed integration checkpoints. |
| Cloud setup or venue network failure during the demo | Identical local cluster as a fallback. |
| Cost of running three cloud machines | Small instances, billing alarm already set, tagged resources and a one-command teardown. |
| Security is prototype-grade | Admin, sync and operator endpoints need a shared secret, but it travels over plain HTTP and anyone can create or read reports. HTTPS, per-user identity and signed messages are the production answer. |
| Not load tested | SQLite per node and three nodes only. The prototype proves behaviour under faults, not throughput. |
| Urgent-first catch-up sync | Not built: it conflicts with the in-order rule that keeps conflict detection correct. We do not claim it. |

## 12. Questions we expect, and our answers

| Question | Answer |
|---|---|
| Why not just use DynamoDB Global Tables? | Its default mode resolves conflicts by latest timestamp, so an edit can vanish silently; its strong mode needs another region to confirm each write, which a cut-off region cannot get. We keep accepting writes and keep every version. |
| What if a region is destroyed? | Anything not yet synced is lost. We minimise that window, mark such reports as local, and say so openly. |
| Why not trust timestamps? | Devices in disaster zones often have wrong clocks, so we use causal information from version vectors and logical clocks. |
| Is this just a CRDT library? | No. We use similar convergence ideas but add disaster-specific rules, an operator ledger, durability status and a built-in verification harness. |
| How do you know it is correct? | Per-rule tests, random multi-node fault runs with invariants, and a comparison against a baseline. |
| Does it scale? | Not tested beyond three nodes and small data. The prototype proves the behaviour, not throughput. |

## Sources


- AWS: DynamoDB Global Tables — aws.amazon.com/dynamodb/global-tables
- AWS: Global tables overview (Prescriptive Guidance) — docs.aws.amazon.com/prescriptive-guidance/latest/dynamodb-global-tables/overview.html
- AWS News Blog: Multi-Region Strong Consistency in DynamoDB global tables
- AWS: Monitoring global tables (ReplicationLatency metric) — docs.aws.eu/amazondynamodb/latest/developerguide/globaltables_monitoring.html
- Amplify docs: DataStore conflict resolution — docs.amplify.aws/flutter/build-a-backend/more-features/datastore/conflict-resolution/
- Kleppmann, M. et al.: Automerge, real-time data sync between edge devices (MobiUK 2018)
- Kleppmann, M.: Local-first software — martin.kleppmann.com/2021/06/14/emergencity.html
- Jepsen: NATS 2.12.1 — jepsen.io/blog/2025-12-08-nats-2.12.1
- Jepsen: MariaDB Galera Cluster 12.1.2 — jepsen.io/blog/2026-03-16-mariadb-galera-cluster-12.1.2
- DZone: Conflict resolution using last-write-wins vs CRDTs


---

Technical claims about other products marked "general knowledge" were not re-verified during our research. Performance figures come from single runs measured from a laptop, on a local cluster and on three AWS regions as labelled.